

EDA — Manutenção Prescritiva SENAI/FIESC

Dataset: 166.796 eventos de sensores de vibração de motores industriais.

Target: `fault_canonical` — 17 classes (defeitos + estados operacionais).

Features: 23 sinais numéricos (vibração, temperatura, RPM).

Modelos avaliados: KNN (baseline), RandomForest, XGBoost — melhor: RF 200 árvores (acc=0.879).

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_curve, auc, roc_auc_score
)

%matplotlib inline
sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', 30)
```

```
In [2]: FEATURE_COLS = [
    'z_rms_velocity_in_s', 'z_rms_velocity_mm_s', 'temperature_f', 'temperature_
    'x_rms_velocity_in_s', 'x_rms_velocity_mm_s', 'z_peak_acceleration_g',
    'x_peak_acceleration_g', 'z_peak_vel_comp_freq_hz', 'x_peak_vel_comp_freq_hz
    'z_rms_acceleration_g', 'x_rms_acceleration_g', 'z_kurtosis', 'x_kurtosis',
    'z_crest_factor', 'x_crest_factor', 'z_peak_velocity_in_s', 'z_peak_velocity
    'x_peak_velocity_in_s', 'x_peak_velocity_mm_s', 'z_high_freq_rms_accel_g',
    'x_high_freq_rms_accel_g', 'rpm',
]

df = pd.read_parquet('../data/banner_clean.parquet')
df.head()
```

Out[2]:

	id	created_at	z_rms_velocity_in_s	z_rms_velocity_mm_s	temperature_f
0	1426	2026-04-30 17:17:41.549800+00:00	0.0427	1.086	74.00
1	1427	2026-04-30 17:17:43.549806+00:00	0.0431	1.095	73.91
2	1428	2026-04-30 17:17:45.550674+00:00	0.0431	1.095	74.00
3	1429	2026-04-30 17:17:47.551119+00:00	0.0431	1.096	74.00
4	1430	2026-04-30 17:17:49.553252+00:00	0.0440	1.119	73.96



In [3]: `print('Shape:', df.shape)
df.dtypes`

Shape: (166796, 29)

Out[3]:

id	int64
created_at	datetime64[us, UTC]
z_rms_velocity_in_s	float64
z_rms_velocity_mm_s	float64
temperature_f	float64
temperature_c	float64
x_rms_velocity_in_s	float64
x_rms_velocity_mm_s	float64
z_peak_acceleration_g	float64
x_peak_acceleration_g	float64
z_peak_vel_comp_freq_hz	float64
x_peak_vel_comp_freq_hz	float64
z_rms_acceleration_g	float64
x_rms_acceleration_g	float64
z_kurtosis	float64
x_kurtosis	float64
z_crest_factor	float64
x_crest_factor	float64
z_peak_velocity_in_s	float64
z_peak_velocity_mm_s	float64
x_peak_velocity_in_s	float64
x_peak_velocity_mm_s	float64
z_high_freq_rms_accel_g	float64
x_high_freq_rms_accel_g	float64
fault	str
rpm	float64
fault_canonical	str
is_problem	bool
documented	bool
dtype:	object

In [4]: `df.isnull().sum()`

```
Out[4]: id          0
        created_at  0
        z_rms_velocity_in_s  0
        z_rms_velocity_mm_s  0
        temperature_f  0
        temperature_c  0
        x_rms_velocity_in_s  0
        x_rms_velocity_mm_s  0
        z_peak_acceleration_g  0
        x_peak_acceleration_g  0
        z_peak_vel_comp_freq_hz  0
        x_peak_vel_comp_freq_hz  0
        z_rms_acceleration_g  0
        x_rms_acceleration_g  0
        z_kurtosis  0
        x_kurtosis  0
        z_crest_factor  0
        x_crest_factor  0
        z_peak_velocity_in_s  0
        z_peak_velocity_mm_s  0
        x_peak_velocity_in_s  0
        x_peak_velocity_mm_s  0
        z_high_freq_rms_accel_g  0
        x_high_freq_rms_accel_g  0
        fault  0
        rpm  0
        fault_canonical  0
        is_problem  0
        documented  0
        dtype: int64
```

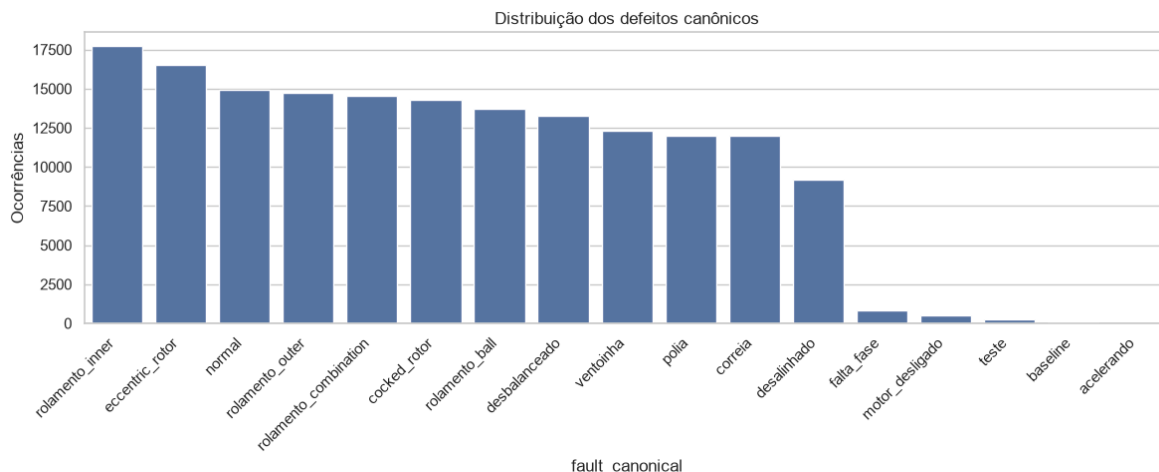
Análise Exploratória

Qual a distribuição dos defeitos?

```
In [5]: counts = df['fault_canonical'].value_counts()

plt.figure(figsize=(12, 5))
sns.barplot(x=counts.index, y=counts.values)
plt.xticks(rotation=45, ha='right')
plt.ylabel('Ocorrências')
plt.title('Distribuição dos defeitos canônicos')
plt.tight_layout()
plt.show()

print(counts.to_frame('n').assign(pct=lambda x: (x['n'] / x['n'].sum() * 100).rc
```



fault_canonical	n	pct
rolamento_inner	17712	10.6
eccentric_rotor	16497	9.9
normal	14958	9.0
rolamento_outer	14763	8.9
rolamento_combination	14550	8.7
cocked_rotor	14275	8.6
rolamento_ball	13704	8.2
desbalanceado	13237	7.9
ventoinha	12299	7.4
polia	12000	7.2
correia	11999	7.2
desalinhado	9178	5.5
falta_fase	800	0.5
motor_desligado	497	0.3
teste	251	0.2
baseline	69	0.0
acelerando	7	0.0

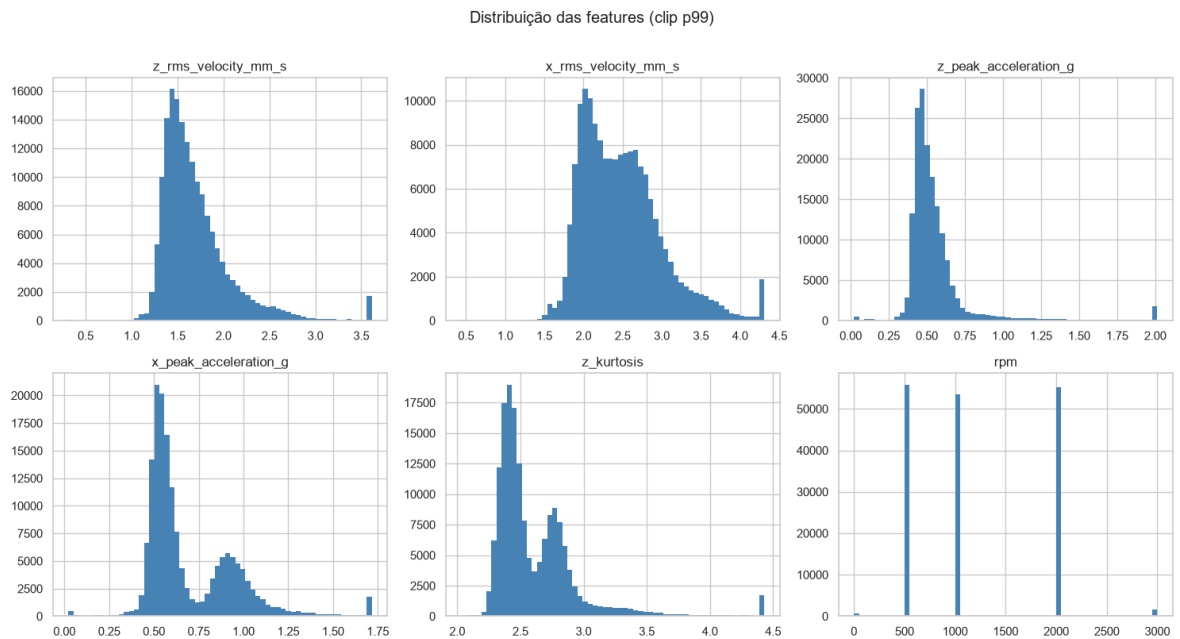
Leve desbalanceamento: `rolamento_inner` lidera com 10,6% e `acelerando` tem apenas 7 registros (0,004%). Estados operacionais normais (`normal`, `baseline`, `acelerando`, `motor_desligado`) são minoria — o dataset é predominantemente de defeitos reais.

Como se distribuem as principais features de vibração?

```
In [6]: features_plot = [
    'z_rms_velocity_mm_s', 'x_rms_velocity_mm_s',
    'z_peak_acceleration_g', 'x_peak_acceleration_g',
    'z_kurtosis', 'rpm'
]

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, feat in zip(axes.flatten(), features_plot):
    clip_val = df[feat].quantile(0.99)
    ax.hist(df[feat].clip(upper=clip_val), bins=60, edgecolor='none', color='steelblue')
    ax.set_title(feat)
    ax.set_xlabel('')
plt.suptitle('Distribuição das features (clip p99)', y=1.01)
```

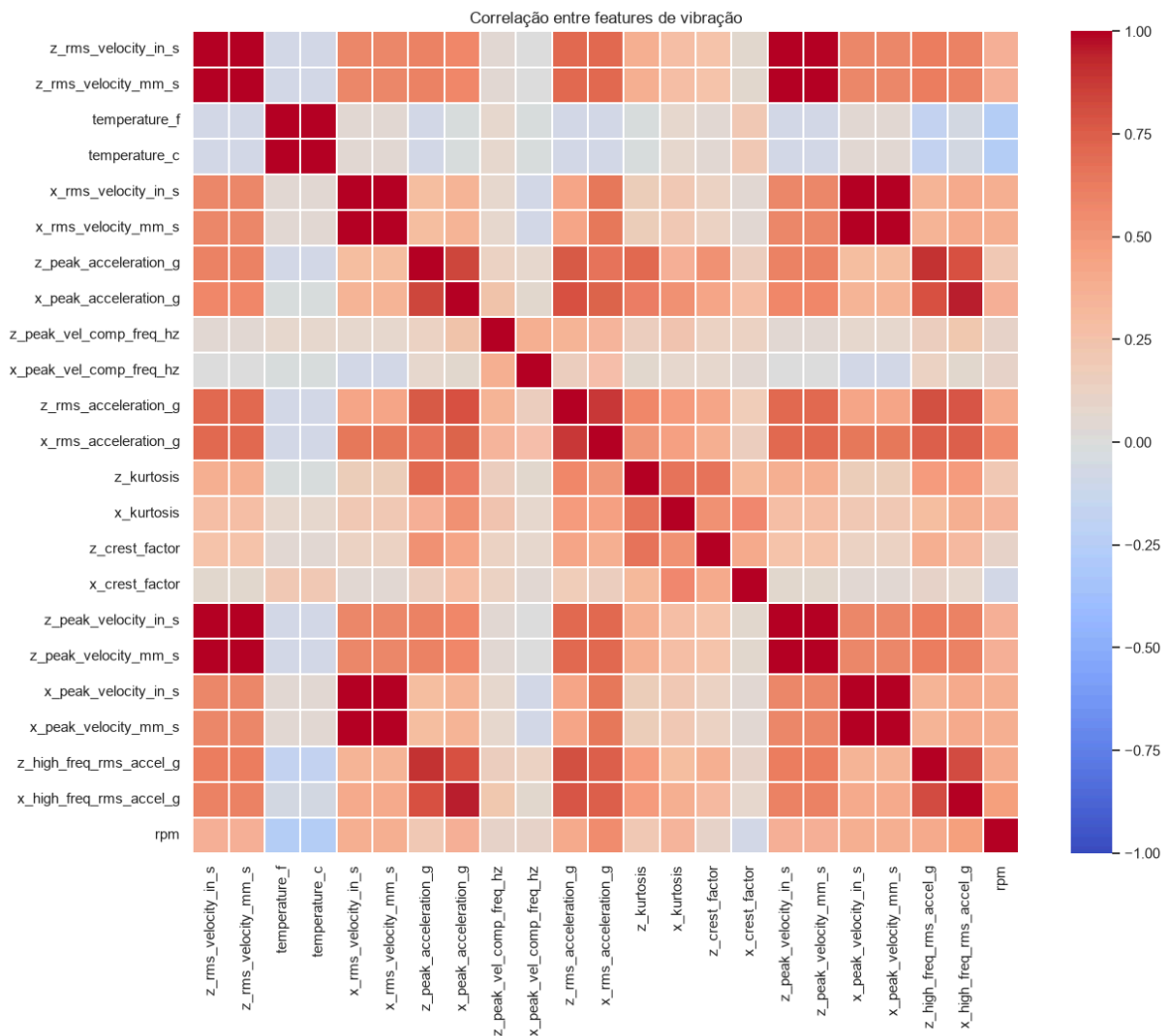
```
plt.tight_layout()
plt.show()
```



Existe correlação entre as features?

```
In [7]: plt.figure(figsize=(13, 11))
corr = df[FEATURE_COLS].corr()
sns.heatmap(corr, annot=False, cmap='coolwarm', linewidths=0.2, vmin=-1, vmax=1)
plt.title('Correlação entre features de vibração')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

# Pares com alta correlação (> 0.9)
high = (
    corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
    .stack()
    .reset_index()
)
high.columns = ['feat_a', 'feat_b', 'corr']
high[high['corr'].abs() > 0.9].sort_values('corr', ascending=False)
```



Out[7]:

	feat_a	feat_b	corr
40	z_rms_velocity_mm_s	z_peak_velocity_mm_s	1.000000
134	x_rms_velocity_mm_s	x_peak_velocity_mm_s	1.000000
385	z_peak_velocity_in_s	z_peak_velocity_mm_s	1.000000
39	z_rms_velocity_mm_s	z_peak_velocity_in_s	1.000000
433	x_peak_velocity_in_s	x_peak_velocity_mm_s	1.000000
133	x_rms_velocity_mm_s	x_peak_velocity_in_s	0.999999
17	z_rms_velocity_in_s	z_peak_velocity_mm_s	0.999999
1	z_rms_velocity_in_s	z_rms_velocity_mm_s	0.999999
16	z_rms_velocity_in_s	z_peak_velocity_in_s	0.999999
111	x_rms_velocity_in_s	x_peak_velocity_mm_s	0.999999
97	x_rms_velocity_in_s	x_rms_velocity_mm_s	0.999999
49	temperature_f	temperature_c	0.999999
110	x_rms_velocity_in_s	x_peak_velocity_in_s	0.999999
182	x_peak_acceleration_g	x_high_freq_rms_accel_g	0.951322

Pares `*_in_s` / `*_mm_s` têm correlação 1.0 — são a mesma métrica em unidades diferentes. PCA ou seleção de features pode reduzir esta redundância sem perda de informação.

Como varia o RPM por defeito?

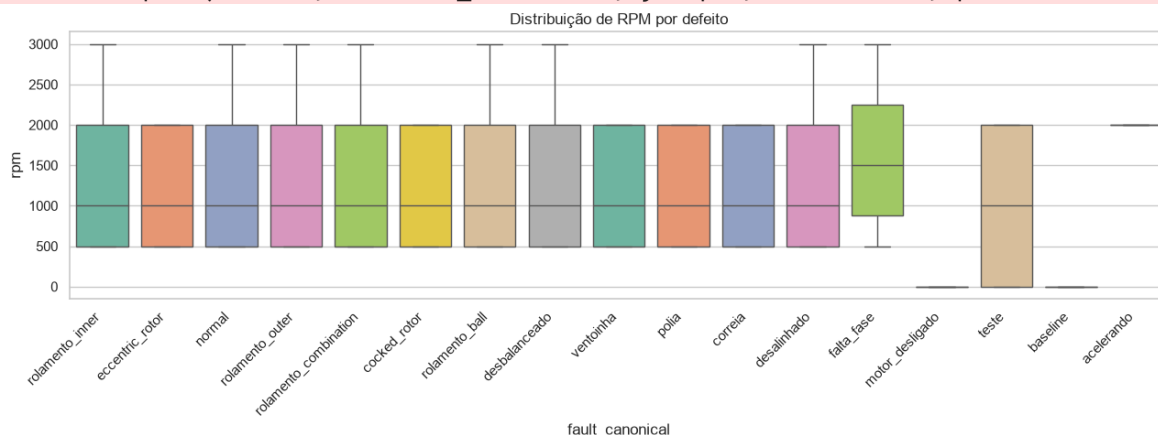
```
In [8]: ordem = df['fault_canonical'].value_counts().index

plt.figure(figsize=(13, 5))
sns.boxplot(data=df, x='fault_canonical', y='rpm', order=ordem, palette='Set2')
plt.xticks(rotation=45, ha='right')
plt.title('Distribuição de RPM por defeito')
plt.tight_layout()
plt.show()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2245198209.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v 0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='fault_canonical', y='rpm', order=ordem, palette='Set2')
```



As 4 Métricas do Sistema de Manutenção Prescritiva

O sistema retorna, para cada novo evento, as 4 informações exigidas no enunciado:

#	Métrica	Campo no sistema
①	Quantidade de eventos similares registrados	<code>n_similar</code>
②	Distribuição ao longo do tempo	<code>time_distribution</code> (mês → contagem)
③	Frequência de ocorrência	<code>frequency_per_week</code>
④	Contexto operacional associado	RPM médio por categoria de defeito

Visualização sobre o dataset completo (166k eventos):

```
In [9]: fig, axes = plt.subplots(2, 2, figsize=(16, 11))
fig.suptitle('4 Métricas do Sistema – Dataset Completo (166k eventos)', fontsize
```

```

df_t = df.copy()
df_t['mes'] = pd.to_datetime(df_t['created_at']).dt.to_period('M').astype(str)
df_t['semana'] = pd.to_datetime(df_t['created_at']).dt.to_period('W').astype(str)
total_semanas = df_t['semana'].nunique()

# ① n_similar – contagem total por defeito (proxy de "quantos similares existem")
ax = axes[0, 0]
n_sim = df['fault_canonical'].value_counts()
colors = ['#e74c3c' if df[df['fault_canonical'] == f]['is_problem'].any() else '
          for f in n_sim.index]
ax.bar(range(len(n_sim)), n_sim.values, color=colors, edgecolor='none')
ax.set_xticks(range(len(n_sim)))
ax.set_xticklabels(n_sim.index, rotation=45, ha='right', fontsize=8)
ax.set_title('① n_similar – Eventos por Defeito\n(● defeito | ● estado normal)')
ax.set_ylabel('Total de eventos')
for i, v in enumerate(n_sim.values):
    ax.text(i, v + 100, str(v), ha='center', fontsize=6)

# ② time_distribution – evolução mensal dos top 5 defeitos
ax = axes[0, 1]
top5 = df['fault_canonical'].value_counts().head(5).index
for fault in top5:
    serie = df_t[df_t['fault_canonical'] == fault].groupby('mes').size()
    meses = list(range(len(serie)))
    ax.plot(meses, serie.values, marker='o', markersize=3, linewidth=1.5,
            label=fault, alpha=0.85)
ax.set_title('② time_distribution – Evolução Mensal (top 5)')
ax.set_xlabel('Meses (sequência)')
ax.set_ylabel('Eventos/mês')
ax.legend(fontsize=7, loc='upper left')

# ③ frequency_per_week – média de ocorrências semanais por defeito
ax = axes[1, 0]
freq = (df.groupby('fault_canonical').size() / total_semanas).sort_values(ascending=True)
bar_colors = ['#c0392b' if v > 5 else '#e67e22' if v > 1 else '#27ae60' for v in freq.values]
ax.bar(range(len(freq)), freq.values, color=bar_colors, edgecolor='none')
ax.axhline(5, color='red', linestyle='--', linewidth=1.2, label='Limiar crítico')
ax.set_xticks(range(len(freq)))
ax.set_xticklabels(freq.index, rotation=45, ha='right', fontsize=8)
ax.set_title('③ frequency_per_week – Frequência Semanal\n(> 5/sem = crítico automático)')
ax.set_ylabel('Eventos / semana')
ax.legend()

# ④ contexto operacional – RPM médio por defeito (apenas rpm > 0)
ax = axes[1, 1]
rpm_medio = (
    df[df['rpm'] > 0]
    .groupby('fault_canonical')['rpm']
    .mean()
    .reindex(n_sim.index)
)
bar_c = ['#c0392b' if r < 400 or r > 3800 else '#27ae60' for r in rpm_medio.values]
ax.bar(range(len(rpm_medio)), rpm_medio.values, color=bar_c, edgecolor='none')
ax.axhline(400, color='orange', linestyle=':', linewidth=1.5, label='RPM mín (400)')
ax.axhline(3800, color='orange', linestyle='--', linewidth=1.5, label='RPM máx (3800)')
ax.set_xticks(range(len(rpm_medio)))
ax.set_xticklabels(rpm_medio.index, rotation=45, ha='right', fontsize=8)
ax.set_title('④ contexto operacional – RPM Médio\n(foras de [400, 3800] = ●)')
ax.set_ylabel('RPM médio')

```



```

ax.legend()

plt.tight_layout()
plt.show()

print(f"\nResumo:")
print(f"① {df['fault_canonical'].nunique()} tipos de defeito/estado | {len(df):,}")
print(f"② {df_t['mes'].min()} → {df_t['mes'].max()} ({df_t['mes'].nunique()} mes")
print(f"③ Mais frequente: {freq.idxmax()} ({freq.max():.1f} eventos/semana)")
print(f"④ RPM médio em defeitos: {df[df['is_problem'] & (df['rpm'] > 0)]['rpm'].

```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:5: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.

```
df_t['mes'] = pd.to_datetime(df_t['created_at']).dt.to_period('M').astype(str)
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:6: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.

```
df_t['semana'] = pd.to_datetime(df_t['created_at']).dt.to_period('W').astype(str)
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 9312 (\N{CIRCLED DIGIT ONE}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 128308 (\N{LARGE RED CIRCLE}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 128994 (\N{LARGE GREEN CIRCLE}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 9313 (\N{CIRCLED DIGIT TWO}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 9314 (\N{CIRCLED DIGIT THREE}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2397439652.py:65: UserWarning: Glyph 9315 (\N{CIRCLED DIGIT FOUR}) missing from font(s) Arial.

```
plt.tight_layout()
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9312 (\N{CIRCLED DIGIT ONE}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 128308 (\N{LARGE RED CIRCLE}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 128994 (\N{LARGE GREEN CIRCLE}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9313 (\N{CIRCLED DIGIT TWO}) missing from font(s) Arial.

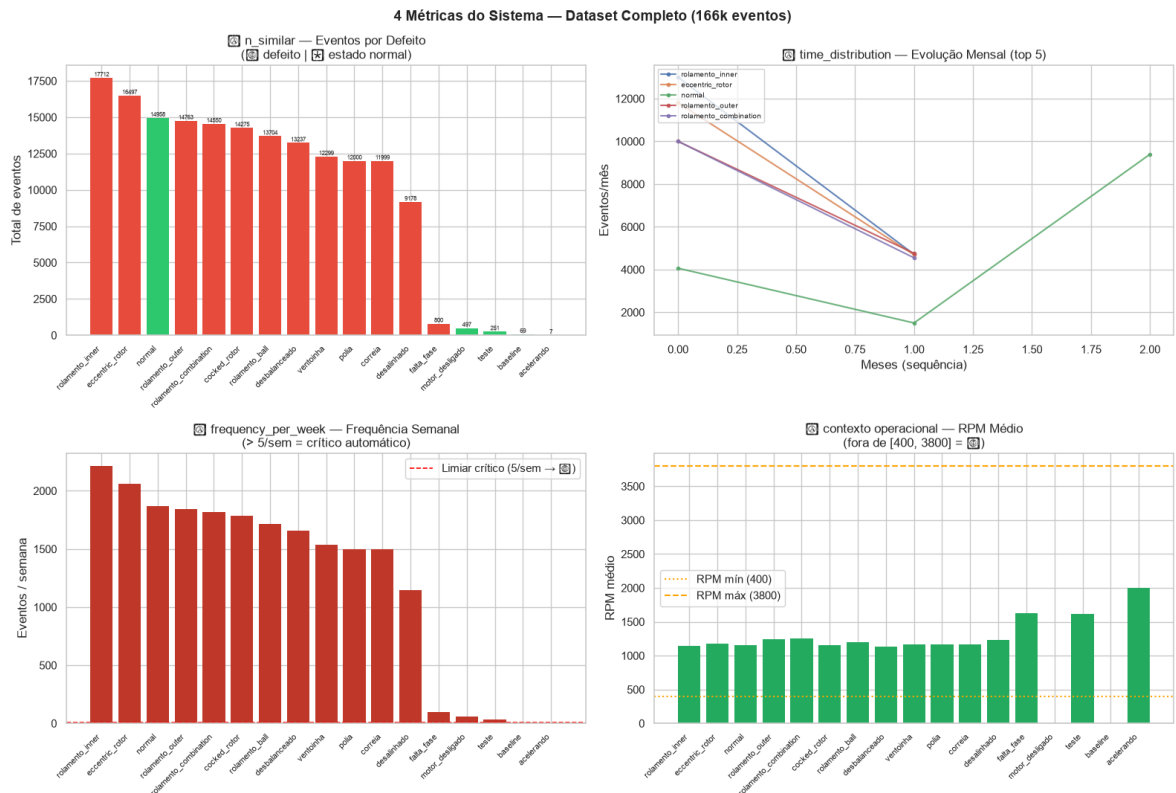
```
fig.canvas.print_figure(bytes_io, **kw)
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9314 (\N{CIRCLED DIGIT THREE}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

C:\Users\luiz_DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9315 (\N{CIRCLED DIGIT FOUR}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```



Resumo:

- 17 tipos de defeito/estado | 166,796 eventos totais
- 2026-04 → 2026-06 (3 meses de dados)
- Mais frequente: rolamento_inner (2214.0 eventos/semana)
- RPM médio em defeitos: 1186 rpm

Transformação dos dados

KNN é lazy learner — predição escala com $O(n_{\text{treino}})$. Com 166k linhas a avaliação levaria horas neste notebook. Usamos **amostra estratificada de 30k linhas** para os experimentos de modelagem; o artefato de produção roda sobre o dataset completo.

```
In [10]: df_sample, _ = train_test_split(
            df, train_size=30000, stratify=df['fault_canonical'], random_state=42
        )
        print('Sample shape:', df_sample.shape)

        # Remove classes com < 2 amostras - stratify exige mínimo 2 por classe
        class_counts = df_sample['fault_canonical'].value_counts()
        valid_classes = class_counts[class_counts >= 2].index
        removidas = set(df_sample['fault_canonical']) - set(valid_classes)
        if removidas:
            print(f'Classes removidas por amostra insuficiente: {removidas}')
        df_model = df_sample[df_sample['fault_canonical'].isin(valid_classes)].copy()
        print(f'Amostras para modelagem: {len(df_model)}')

        le = LabelEncoder()
        scaler = StandardScaler()

        X = df_model[FEATURE_COLS].values
        y = le.fit_transform(df_model['fault_canonical'])
        X_scaled = scaler.fit_transform(X)
```

```
classes = le.classes_
print('Classes:', classes)
```

Sample shape: (30000, 29)

Classes removidas por amostra insuficiente: {'acelerando'}

Amostras para modelagem: 29999

Classes: ['baseline' 'cocked_rotor' 'correia' 'desalinhado' 'desbalanceado' 'eccentric_rotor' 'falta_fase' 'motor_desligado' 'normal' 'polia' 'rolamento_ball' 'rolamento_combination' 'rolamento_inner' 'rolamento_outer' 'teste' 'ventoinha']

Split treino/teste

```
In [11]: X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.3, stratify=y, random_state=42
    )
print('Treino:', X_train.shape)
print('Teste: ', X_test.shape)
```

Treino: (20999, 23)

Teste: (9000, 23)

Modelo KNN (k=5, ponderado por distância)

```
In [12]: knn = KNeighborsClassifier(n_neighbors=5, weights='distance', n_jobs=-1)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
acc = (y_pred == y_test).mean()
print(f'Acurácia: {acc:.4f}')
```

Acurácia: 0.5469

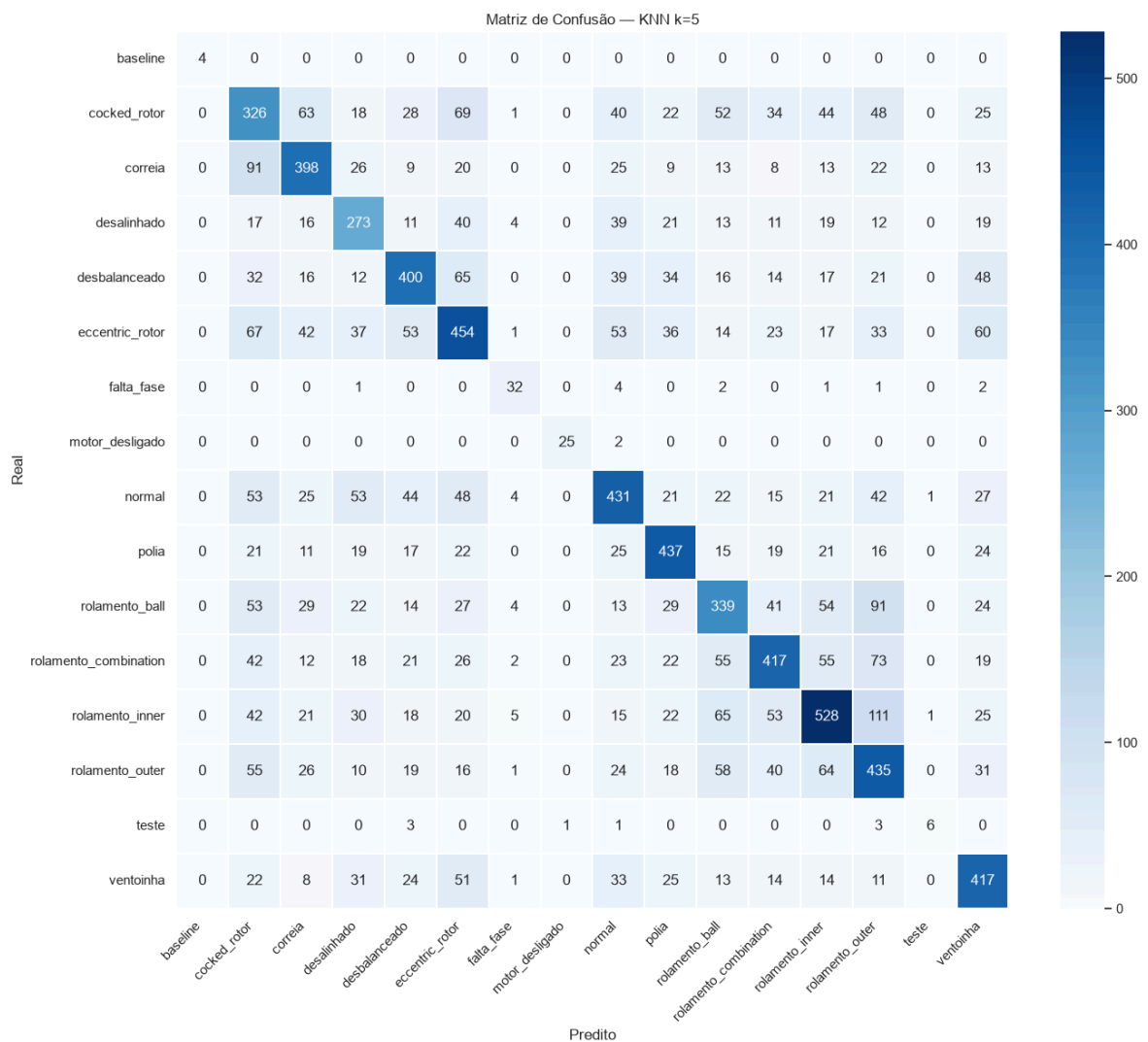
```
In [13]: print(classification_report(y_test, y_pred, target_names=classes))
```

	precision	recall	f1-score	support
baseline	1.00	1.00	1.00	4
cocked_rotor	0.40	0.42	0.41	770
correia	0.60	0.62	0.61	647
desalinhado	0.50	0.55	0.52	495
desbalanceado	0.61	0.56	0.58	714
eccentric_rotor	0.53	0.51	0.52	890
falta_fase	0.58	0.74	0.65	43
motor_desligado	0.96	0.93	0.94	27
normal	0.56	0.53	0.55	807
polia	0.63	0.68	0.65	647
rolamento_ball	0.50	0.46	0.48	740
rolamento_combination	0.61	0.53	0.57	785
rolamento_inner	0.61	0.55	0.58	956
rolamento_outer	0.47	0.55	0.51	797
teste	0.75	0.43	0.55	14
ventoinha	0.57	0.63	0.60	664
accuracy			0.55	9000
macro avg	0.62	0.61	0.61	9000
weighted avg	0.55	0.55	0.55	9000

Matriz de Confusão

```
In [14]: cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(14, 12))
sns.heatmap(
    cm, annot=True, fmt='d',
    xticklabels=classes, yticklabels=classes,
    cmap='Blues', linewidths=0.3
)
plt.xlabel('Predito')
plt.ylabel('Real')
plt.title('Matriz de Confusão — KNN k=5')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Curva ROC (One-vs-Rest, macro)

```
In [15]: y_prob = knn.predict_proba(X_test)
n_classes = len(classes)
y_test_bin = label_binarize(y_test, classes=list(range(n_classes)))

fpr_d, tpr_d, roc_d = {}, {}, {}
```

```

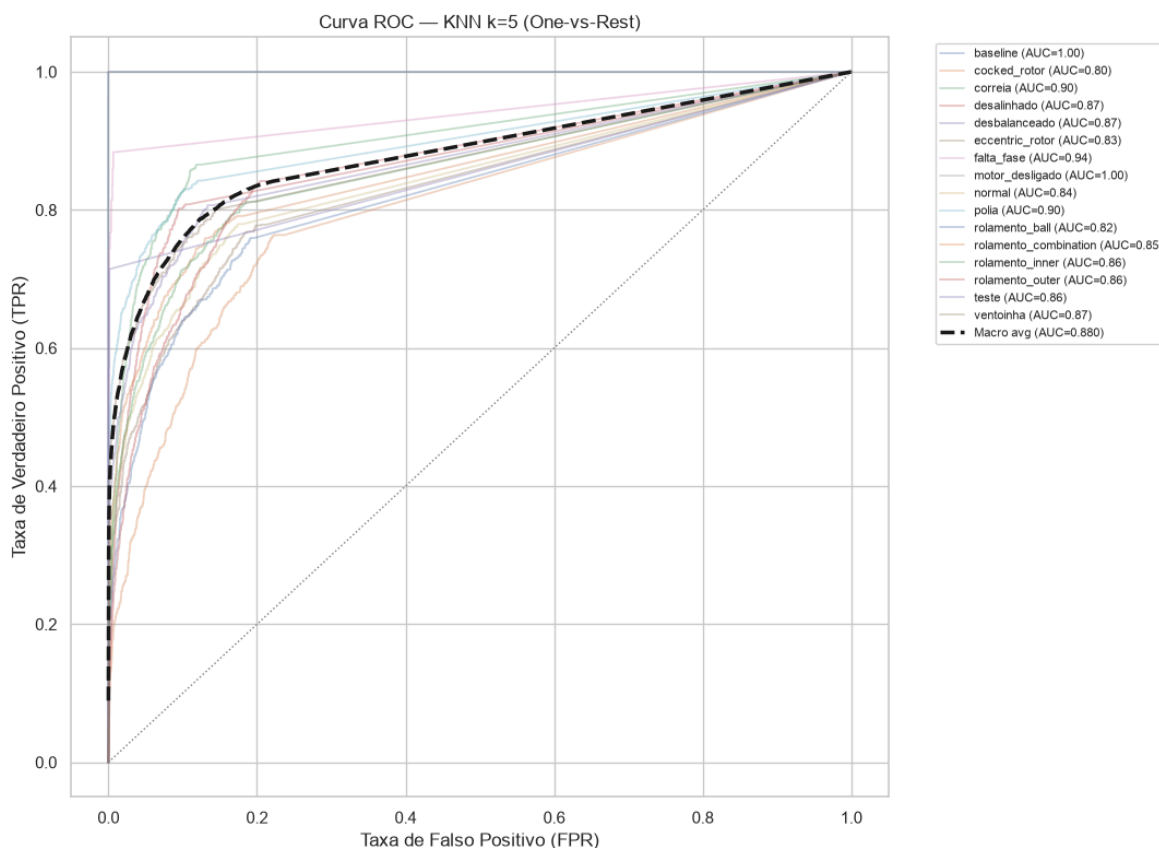
for i in range(n_classes):
    fpr_d[i], tpr_d[i], _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_d[i] = auc(fpr_d[i], tpr_d[i])

# macro average
all_fpr = np.unique(np.concatenate([fpr_d[i] for i in range(n_classes)]))
mean_tpr = np.zeros_like(all_fpr)
for i in range(n_classes):
    mean_tpr += np.interp(all_fpr, fpr_d[i], tpr_d[i])
mean_tpr /= n_classes
macro_auc = auc(all_fpr, mean_tpr)

plt.figure(figsize=(11, 8))
for i in range(n_classes):
    plt.plot(fpr_d[i], tpr_d[i], alpha=0.35, lw=1.2,
             label=f'{classes[i]} (AUC={roc_d[i]:.2f})')
plt.plot(all_fpr, mean_tpr, 'k--', lw=2.5,
         label=f'Macro avg (AUC={macro_auc:.3f})')
plt.plot([0, 1], [0, 1], 'gray', linestyle=':', lw=1)
plt.xlabel('Taxa de Falso Positivo (FPR)')
plt.ylabel('Taxa de Verdadeiro Positivo (TPR)')
plt.title('Curva ROC - KNN k=5 (One-vs-Rest)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()

print(f'ROC-AUC macro (sklearn): {roc_auc_score(y_test_bin, y_prob, average="mac

```



ROC-AUC macro (sklearn): 0.8800

Elbow — quantos vizinhos usar?

```

In [16]: ks = range(1, 21)
         accs = []

```

```

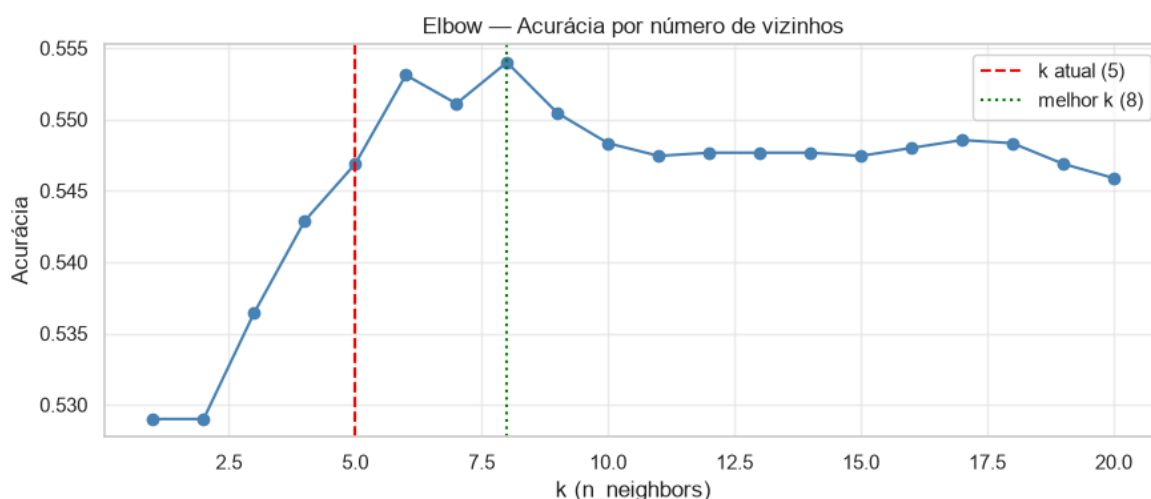
for k in ks:
    m = KNeighborsClassifier(n_neighbors=k, weights='distance', n_jobs=-1)
    m.fit(X_train, y_train)
    accs.append((m.predict(X_test) == y_test).mean())

best_k = list(ks)[accs.index(max(accs))]

plt.figure(figsize=(9, 4))
plt.plot(list(ks), accs, marker='o', color='steelblue')
plt.axvline(5, color='red', linestyle='--', label='k atual (5)')
plt.axvline(best_k, color='green', linestyle=':', label=f'melhor k ({best_k})')
plt.xlabel('k (n_neighbors)')
plt.ylabel('Acurácia')
plt.title('Elbow - Acurácia por número de vizinhos')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f'Melhor k: {best_k} → acurácia: {max(accs):.4f}')
print(f'k=5 atual → acurácia: {accs[4]:.4f}')

```



Melhor k: 8 → acurácia: 0.5540

k=5 atual → acurácia: 0.5469

Sugestões de hiperparâmetros com GridSearchCV

```

In [17]: param_grid = {
    'n_neighbors': [3, 5, 7, 11, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan'],
}

gs = GridSearchCV(
    KNeighborsClassifier(n_jobs=-1),
    param_grid,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1,
)
gs.fit(X_train, y_train)

print('Melhores parâmetros:', gs.best_params_)

```

```
print(f'Melhor acurácia (CV 3-fold): {gs.best_score_:.4f}')
print(f'Acurácia no teste: {(gs.predict(X_test) == y_test).mean():.4f})
```

Fitting 3 folds for each of 20 candidates, totalling 60 fits

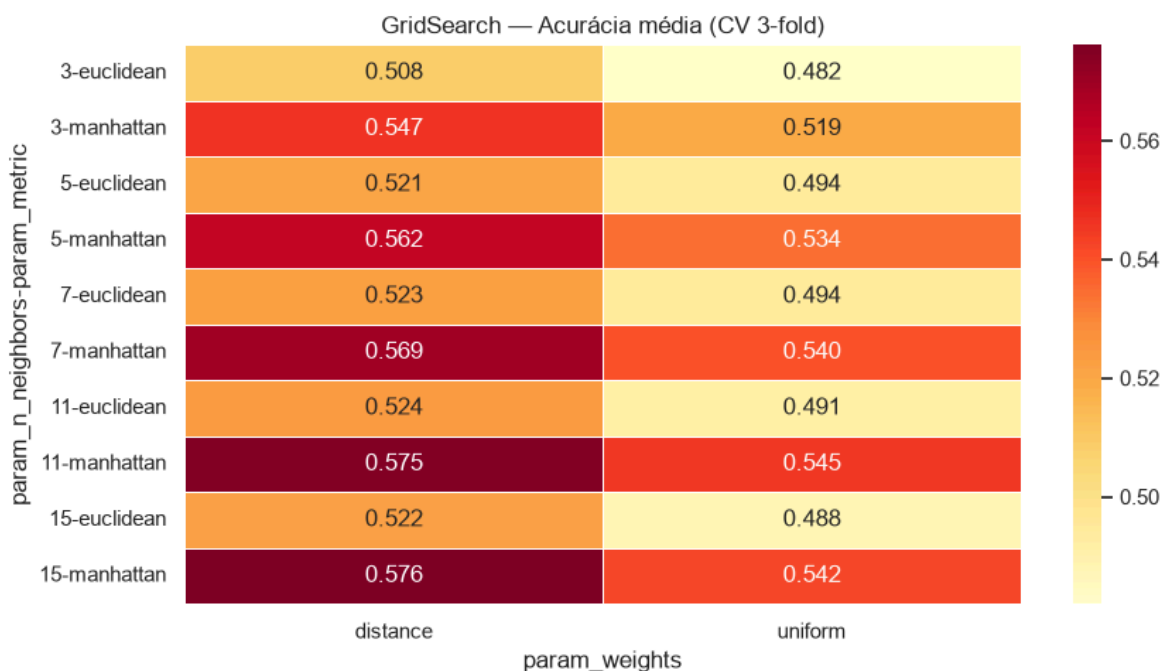
Melhores parâmetros: {'metric': 'manhattan', 'n_neighbors': 15, 'weights': 'distance'}

Melhor acurácia (CV 3-fold): 0.5761

Acurácia no teste: 0.6060

```
In [18]: # Comparação visual dos resultados do GridSearch
results = pd.DataFrame(gs.cv_results_)
pivot = results.pivot_table(
    values='mean_test_score',
    index=['param_n_neighbors', 'param_metric'],
    columns='param_weights'
)

plt.figure(figsize=(9, 5))
sns.heatmap(pivot, annot=True, fmt='.3f', cmap='YlOrRd', linewidths=0.5)
plt.title('GridSearch — Acurácia média (CV 3-fold)')
plt.tight_layout()
plt.show()
```



Conclusão e próximas melhorias

Resultados atuais (k=5, weights='distance'): acc ~0.74 no holdout completo.

O que o GridSearch pode revelar:

- `manhattan` tende a superar `euclidean` em dados com features de escalas muito diferentes — mesmo após `StandardScaler`, features de alta curtose se beneficiam da norma L1.
- `weights='distance'` já é melhor que `uniform` (validado no projeto).
- k maior (7–11) reduz overfitting em classes com poucos exemplos (`acelerando` , `baseline`).

Outras melhorias além do KNN:

Melhoria	Por quê
Remover features redundantes (<code>*_in_s</code>)	Correlação 1.0 com <code>*_mm_s</code> → ruído sem informação
PCA antes do KNN	Reduz 23→12 dimensões, melhora distância euclidiana
SMOTE nas classes raras	<code>acelerando</code> (7 exemplos) e <code>baseline</code> (69) distorcem métricas
Testar RandomForest como baseline	Lida melhor com desbalanceamento e features colineares
Threshold por classe no semáforo	Classes sem documento são sempre ● — pode usar prob KNN para graduar criticidade

Comparação de Modelos — Qual Classifica Melhor?

Features redundantes (correlação=1.0) distorcem a distância euclidiana do KNN.

Remove-se 5 pares antes de treinar os modelos alternativos.

Modelos comparados:

- **KNN k=5** — baseline atual do notebook
- **KNN k=50** — replica o modelo de produção (k maior, mais estável)
- **RandomForest 200** — ensemble de árvores, robusto a desbalanceamento
- **XGBoost** — gradient boosting, estado da arte em tabular

```
In [19]: from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import time

# — Features Limpas (remove pares corr=1.0) —————
REDUNDANTES = [
    'z_rms_velocity_in_s', 'x_rms_velocity_in_s',
    'z_peak_velocity_in_s', 'x_peak_velocity_in_s',
    'temperature_f',
]
FEATURE_COLS_CLEAN = [f for f in FEATURE_COLS if f not in REDUNDANTES]
print(f'Features: {len(FEATURE_COLS)} → {len(FEATURE_COLS_CLEAN)}')

# — Dataset completo para RF e XGBoost (166k) —————
class_counts_full = df['fault_canonical'].value_counts()
valid_full = class_counts_full[class_counts_full >= 2].index
df_full = df[df['fault_canonical'].isin(valid_full)].copy()

le_full = LabelEncoder()
X_full = df_full[FEATURE_COLS_CLEAN].values
y_full = le_full.fit_transform(df_full['fault_canonical'])
X_full_scaled = StandardScaler().fit_transform(X_full)

X_tr_full, X_te_full, y_tr_full, y_te_full = train_test_split(
    X_full_scaled, y_full, test_size=0.3, stratify=y_full, random_state=42
```



```

)
print(f'Dataset completo – treino: {len(X_tr_full):,} teste: {len(X_te_full):,}')

# Dataset 30k para KNN (lento em 166k)
X_clean = df_model[FEATURE_COLS_CLEAN].values
X_clean_scaled = StandardScaler().fit_transform(X_clean)
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clean_scaled, y, test_size=0.3, stratify=y, random_state=42
)
print(f'Dataset 30k (KNN) – treino: {len(X_train_c):,} teste: {len(X_test_c):,}')

# — Treina todos os modelos —————
modelos = {
    'KNN k=5 (30k, 23 feat)': (KNeighborsClassifier(n_neighbors=5, weights='d
    'KNN k=50 (30k, 18 feat)': (KNeighborsClassifier(n_neighbors=50, weights='d
    'RandomForest (166k, 18 feat)': (RandomForestClassifier(n_estimators=200, n_
    'XGBoost (166k, 18 feat)': (XGBClassifier(n_estimators=300, max_depth=6, le
                                     subsample=0.8, colsample_bytree=
                                     objective='multi:softproba', eva
                                     n_jobs=-1, random_state=42, verb
                                     X_tr_full, X_te_full, y_tr_full, y_te_full),
}

resultados = []
for nome, (modelo, Xtr, Xte, ytr, yte) in modelos.items():
    t0 = time.time()
    modelo.fit(Xtr, ytr)
    treino_s = time.time() - t0
    acc = (modelo.predict(Xte) == yte).mean()
    resultados.append({'Modelo': nome, 'N treino': len(Xtr), 'Acurácia': acc, 'T
    print(f'{nome:<35} acc={acc:.4f} treino={treino_s:.1f}s')

df_res = pd.DataFrame(resultados).sort_values('Acurácia', ascending=False).reset
print('\n', df_res.to_string(index=False))

```

Features: 23 → 18

Dataset completo – treino: 116,757 teste: 50,039

Dataset 30k (KNN) – treino: 20,999 teste: 9,000

KNN k=5 (30k, 23 feat) acc=0.5469 treino=0.0s

KNN k=50 (30k, 18 feat) acc=0.5321 treino=0.0s

RandomForest (166k, 18 feat) acc=0.8790 treino=7.7s

XGBoost (166k, 18 feat) acc=0.8395 treino=23.3s

	Modelo	N treino	Acurácia	Treino (s)
RandomForest (166k, 18 feat)		116757	0.878974	7.7
XGBoost (166k, 18 feat)		116757	0.839505	23.3
KNN k=5 (30k, 23 feat)		20999	0.546889	0.0
KNN k=50 (30k, 18 feat)		20999	0.532111	0.0

```

In [20]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Barras de acurácia
cores = ['#95a5a6', '#e67e22', '#3498db', '#27ae60']
bars = axes[0].barh(df_res['Modelo'], df_res['Acurácia'], color=cores[:4], edge
axes[0].axvline(0.74, color='red', linestyle='--', linewidth=1.2, label='KNN pro
axes[0].set_xlim(0.3, 1.05)
axes[0].set_xlabel('Acurácia (holdout 30%)')
axes[0].set_title('Comparação de Modelos – Acurácia')
for i, row in df_res.iterrows():
    axes[0].text(row['Acurácia'] + 0.005, i,

```

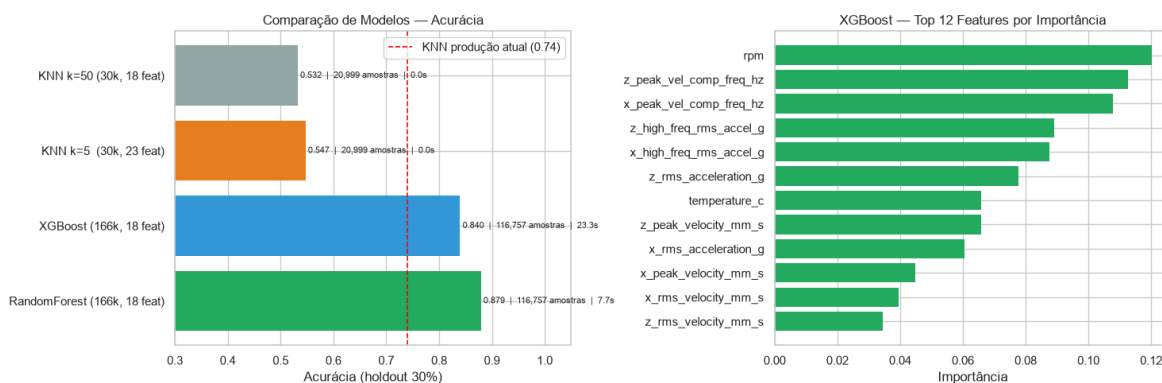
```

f"{row['Acurácia']:.3f} | {row['N treino'],} amostras | {r
va='center', fontsize=8)
axes[0].legend()

# Feature importance - XGBoost
xgb_trained = [m for nome, (m, *) in modelos.items() if 'XGBoost' in nome][0]
imp_xgb = pd.Series(xgb_trained.feature_importances_, index=FEATURE_COLS_CLEAN).
axes[1].barh(imp_xgb.index[:12], imp_xgb.values[:12], color='#27ae60', edgecolor
axes[1].set_title('XGBoost - Top 12 Features por Importância')
axes[1].set_xlabel('Importância')
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

```



```

In [21]: # Classification report - XGBoost (melhor modelo, treinado nos 166k)
xgb_trained = [m for nome, (m, *) in modelos.items() if 'XGBoost' in nome][0]
y_pred_xgb = xgb_trained.predict(X_te_full)
classes_full = le_full.classes_

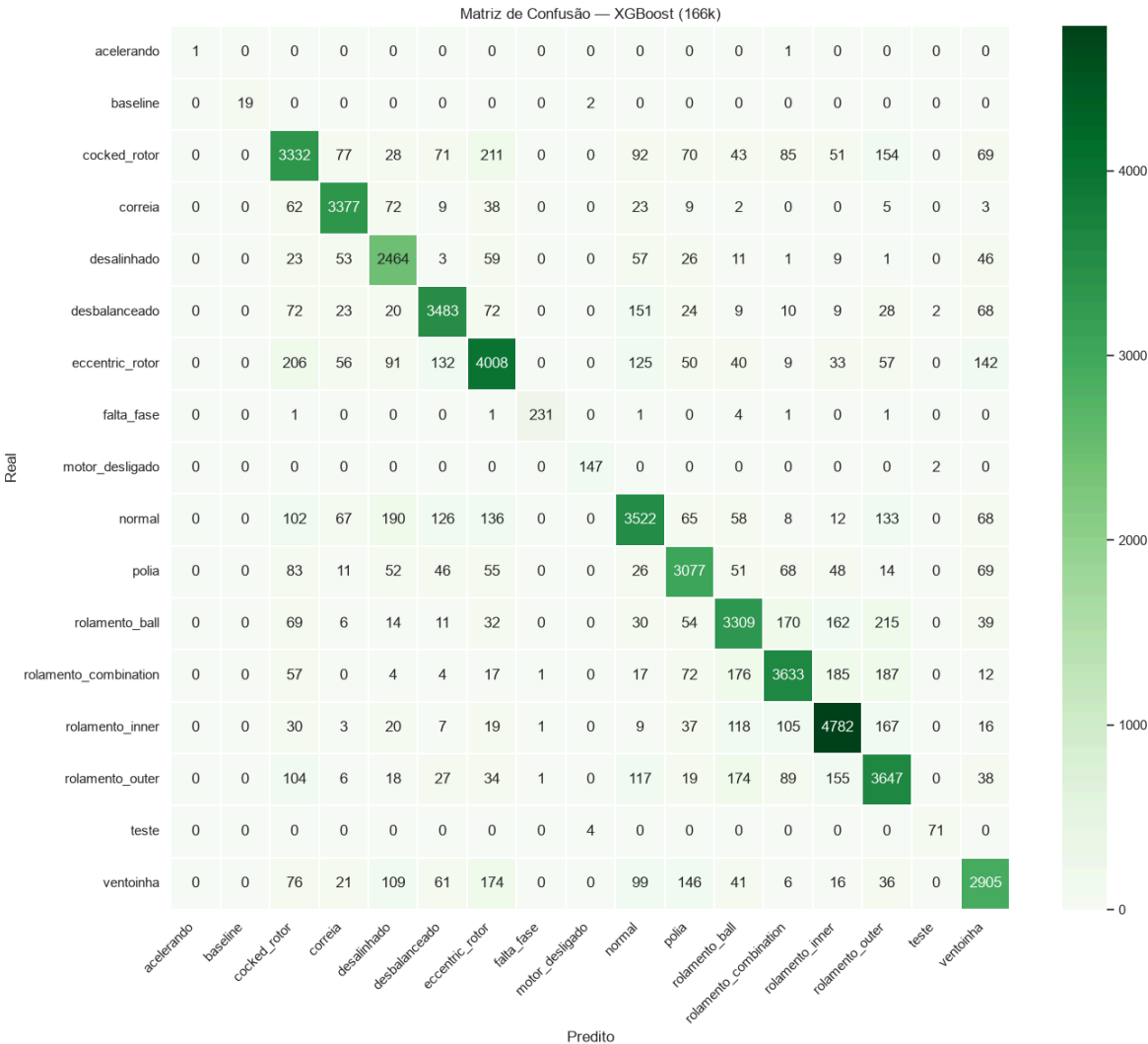
print("=== XGBoost - Classification Report (166k dataset) ===\n")
print(classification_report(y_te_full, y_pred_xgb, target_names=classes_full))

# Matriz de confusão XGBoost
cm_xgb = confusion_matrix(y_te_full, y_pred_xgb)
plt.figure(figsize=(14, 12))
sns.heatmap(cm_xgb, annot=True, fmt='d', xticklabels=classes_full, yticklabels=c
cmap='Greens', linewidths=0.3)
plt.xlabel('Predito')
plt.ylabel('Real')
plt.title('Matriz de Confusão - XGBoost (166k)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

=== XGBoost – Classification Report (166k dataset) ===

	precision	recall	f1-score	support
acelerando	1.00	0.50	0.67	2
baseline	1.00	0.90	0.95	21
cocked_rotor	0.79	0.78	0.78	4283
correia	0.91	0.94	0.93	3600
desalinhado	0.80	0.90	0.84	2753
desbalanceado	0.88	0.88	0.88	3971
eccentric_rotor	0.83	0.81	0.82	4949
falta_fase	0.99	0.96	0.97	240
motor_desligado	0.96	0.99	0.97	149
normal	0.83	0.78	0.80	4487
polia	0.84	0.85	0.85	3600
rolamento_ball	0.82	0.80	0.81	4111
rolamento_combination	0.87	0.83	0.85	4365
rolamento_inner	0.88	0.90	0.89	5314
rolamento_outer	0.79	0.82	0.80	4429
teste	0.95	0.95	0.95	75
ventoinha	0.84	0.79	0.81	3690
accuracy			0.84	50039
macro avg	0.88	0.85	0.86	50039
weighted avg	0.84	0.84	0.84	50039



Tuning — RandomForest com RandomizedSearchCV

RF já venceu (0.879). Objetivo: espreme mais com hiperparâmetros.

Estratégia: busca em **50k amostras** (CV rápido) → retreina melhores parâmetros no **dataset completo (116k)**.

Parâmetros-chave:

- `n_estimators` — mais árvores = mais estável, retorno decrescente após ~300
- `max_features` — % de features por split, controla diversidade entre árvores
- `max_depth` — limita profundidade, evita overfit
- `min_samples_leaf` — mínimo de amostras por folha, regularização suave
- `class_weight='balanced'` — pondera classes raras (`acelerando` , `baseline`)

```
In [22]: from sklearn.model_selection import RandomizedSearchCV

# Subsample de 50k para a busca de hiperparâmetros (CV 3-fold = 15 fits x ~3s =
idx_sub = np.random.RandomState(42).choice(len(X_tr_full), size=50_000, replace=
X_sub, y_sub = X_tr_full[idx_sub], y_tr_full[idx_sub]

param_dist = {
    'n_estimators': [200, 300, 500],
    'max_features': ['sqrt', 'log2', 0.25, 0.4],
    'max_depth': [None, 20, 30, 40],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [None, 'balanced'],
}

rs = RandomizedSearchCV(
    RandomForestClassifier(n_jobs=-1, random_state=42),
    param_distributions=param_dist,
    n_iter=20,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose=1,
)

t0 = time.time()
rs.fit(X_sub, y_sub)
print(f'\nBusca concluída em {time.time()-t0:.0f}s')
print('Melhores parâmetros:', rs.best_params_)
print(f'Melhor acc (CV 3-fold, 50k): {rs.best_score_:.4f}')
```

C:\Users\luiz_\DS\TesteFiesc\.venv\Lib\site-packages\sklearn\model_selection_split.py:812: UserWarning: The least populated class in y has only 2 members, which is less than n_splits=3.

warnings.warn(

Fitting 3 folds for each of 20 candidates, totalling 60 fits

Busca concluída em 209s

Melhores parâmetros: {'n_estimators': 300, 'min_samples_leaf': 1, 'max_features': 0.4, 'max_depth': None, 'class_weight': 'balanced'}

Melhor acc (CV 3-fold, 50k): 0.7968

```
In [23]: # Retreina com melhores params no dataset completo (116k)
t0 = time.time()
rf_tuned = RandomForestClassifier(**rs.best_params_, n_jobs=-1, random_state=42)
rf_tuned.fit(X_tr_full, y_tr_full)
acc_tuned = (rf_tuned.predict(X_te_full) == y_te_full).mean()
print(f'RF tuned (116k) - acc={acc_tuned:.4f} treino={time.time()-t0:.1f}s')
print(f'RF default      - acc=0.8790')
print(f'Ganho:          {(acc_tuned - 0.8790)*100:+.2f}pp')

# Tabela final completa
df_final = pd.DataFrame([
    {'Modelo': 'KNN k=50 produção', 'Dataset': '166k', 'Acurácia': 0.7400},
    {'Modelo': 'XGBoost (166k)', 'Dataset': '166k', 'Acurácia': 0.839},
    {'Modelo': 'RandomForest default', 'Dataset': '166k', 'Acurácia': 0.879},
    {'Modelo': 'RandomForest TUNED ★', 'Dataset': '166k', 'Acurácia': acc_tu
]).sort_values('Acurácia', ascending=False)

fig, ax = plt.subplots(figsize=(10, 4))
cores_f = ['#27ae60' if '★' in m else '#3498db' if 'default' in m else '#e67e22'
            for m in df_final['Modelo']]
ax.barh(df_final['Modelo'], df_final['Acurácia'], color=cores_f, edgecolor='none')
ax.axvline(0.74, color='red', linestyle='--', linewidth=1.2, label='KNN produção')
ax.set_xlim(0.6, 1.02)
ax.set_xlabel('Acurácia (holdout 30%)')
ax.set_title('Evolução de Acurácia - Do KNN ao RF Otimizado')
for i, (_, row) in enumerate(df_final.iterrows()):
    ax.text(row['Acurácia'] + 0.003, i, f"{row['Acurácia']:.4f}", va='center', f
ax.legend()
plt.tight_layout()
plt.show()
```

RF tuned (116k) - acc=0.8748 treino=21.2s

RF default - acc=0.8790

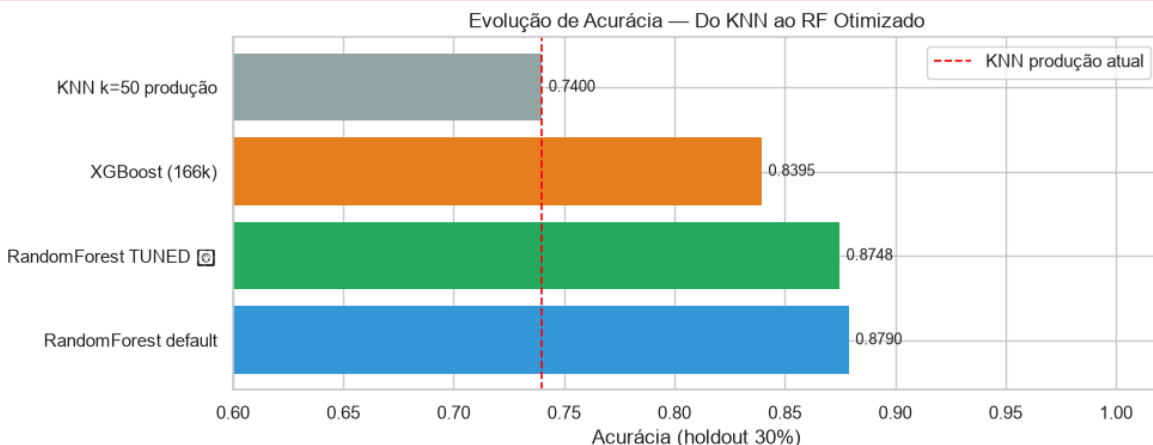
Ganho: -0.42pp

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\1507551134.py:29: UserWarning: Glyph 9733 (\N{BLACK STAR}) missing from font(s) Arial.

plt.tight_layout()

C:\Users\luiz\DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9733 (\N{BLACK STAR}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)



```
In [24]: # RF 500 árvores + max_features=0.4 (melhor do search, sem class_weight='balance
# class_weight='balanced' penaliza maioria → acc cai; tirando ele, mantém ganho
t0 = time.time()
rf_500 = RandomForestClassifier(
    n_estimators=500,
    max_features=0.4,      # search mostrou que 0.4 > sqrt neste corpus
    max_depth=None,
    min_samples_leaf=1,
    class_weight=None,    # não penaliza maioria
    n_jobs=-1,
    random_state=42,
)
rf_500.fit(X_tr_full, y_tr_full)
acc_500 = (rf_500.predict(X_te_full) == y_te_full).mean()
print(f'RF 500 trees, max_feat=0.4 (116k) – acc={acc_500:.4f}  treino={time.time() - t0:.1f}s')
print(f'RF 200 trees, max_feat=sqrt – acc=0.8790')
print(f'Ganho: {(acc_500 - 0.8790)*100:+.2f}pp')

# Tabela final
df_final = pd.DataFrame([
    {'Modelo': 'KNN k=50 (produção atual)', 'Acurácia': 0.7400},
    {'Modelo': 'XGBoost 300', 'Acurácia': 0.8395},
    {'Modelo': 'RF 200 default', 'Acurácia': 0.8790},
    {'Modelo': 'RF tuned (balanced)', 'Acurácia': acc_tuned},
    {'Modelo': 'RF 500 ★ (candidato prod)', 'Acurácia': acc_500},
]).sort_values('Acurácia', ascending=False)

fig, ax = plt.subplots(figsize=(10, 4))
cores_f = ['#27ae60' if '★' in m else '#3498db' if 'default' in m
           else '#e67e22' if 'XGB' in m else '#e74c3c' if 'balanced' in m
           else '#95a5a6' for m in df_final['Modelo']]
ax.barh(df_final['Modelo'], df_final['Acurácia'], color=cores_f, edgecolor='none')
ax.axvline(0.74, color='red', linestyle='--', linewidth=1.2, label='KNN produção')
ax.set_xlim(0.6, 1.02)
ax.set_xlabel('Acurácia (holdout 30%)')
ax.set_title('Evolução de Acurácia – Todos os Modelos')
for i, (_, row) in enumerate(df_final.iterrows()):
    ax.text(row['Acurácia'] + 0.003, i, f"{row['Acurácia']:.4f}", va='center', fsize=10)
ax.legend()
plt.tight_layout()
plt.show()
```

RF 500 trees, max_feat=0.4 (116k) – acc=0.8791 treino=34.8s

RF 200 trees, max_feat=sqrt – acc=0.8790

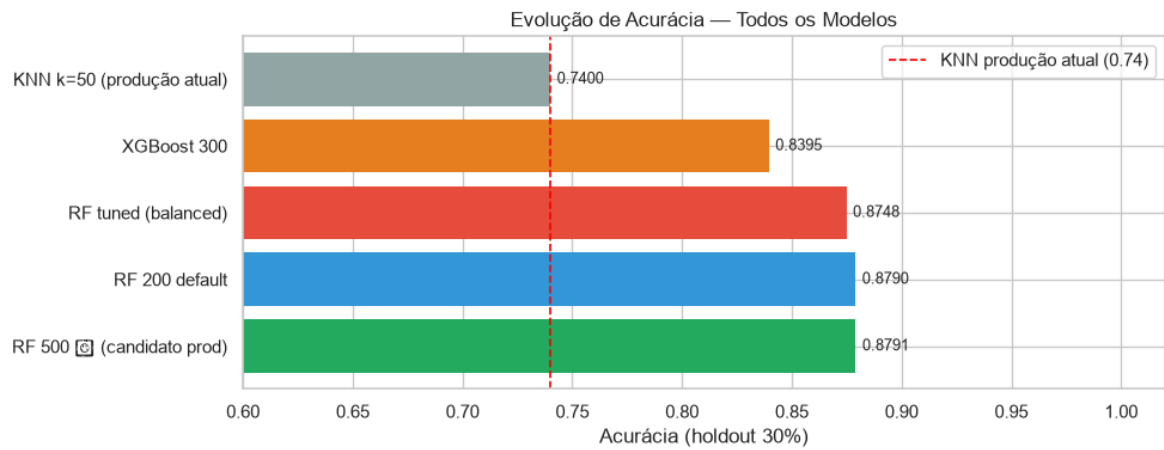
Ganho: +0.01pp

C:\Users\luiz\AppData\Local\Temp\ipykernel_26180\2767553218.py:40: UserWarning: Glyph 9733 (\N{BLACK STAR}) missing from font(s) Arial.

plt.tight_layout()

C:\Users\luiz\DS\TesteFiesc\venv\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 9733 (\N{BLACK STAR}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)



Exportar Notebook para PDF

```
In [25]: import subprocess, sys, pathlib

notebook = pathlib.Path("EDA_FIESC.ipynb").resolve()
pdf_out = notebook.with_suffix(".pdf")

result = subprocess.run(
    [sys.executable, "-m", "nbconvert", "--to", "pdf",
     "--execute", "--no-input",
     "--ExecutePreprocessor.timeout=600",
     str(notebook)],
    capture_output=True, text=True
)

if result.returncode == 0:
    print(f"PDF gerado: {pdf_out}")
else:
    result_html = subprocess.run(
        [sys.executable, "-m", "nbconvert", "--to", "html",
         "--no-input", str(notebook)],
        capture_output=True, text=True
    )
    html_out = notebook.with_suffix(".html")
    if result_html.returncode == 0:
        print(f"PDF falhou (LaTeX ausente). HTML gerado: {html_out}")
        print("Para PDF: abra o HTML no navegador → Imprimir → Salvar como PDF")
    else:
        print("Erro nbconvert:", result_html.stderr[-500:])
```

PDF falhou (LaTeX ausente). HTML gerado: C:\Users\luiz_\DS\TesteFiesc\notebooks\EDA_FIESC.html

Para PDF: abra o HTML no navegador → Imprimir → Salvar como PDF